

GTM SYSTEMS • PLATFORM & DEVOPS

DevOps, Code Repository and Jira Operating Model

SALDEV-1500 — source control, CI/CD, deployment governance and implementation backlog

Document control	Value
Status	Proposed operating standard — approval and implementation required
Prepared for	Flywire GTM Systems, Platform & DevOps
Prepared on	28 August 2026
Decision requested	Approve target toolchain, repository standard, Jira workflow and implementation backlog
Scope	Salesforce and adjacent GTM systems; configuration, code, APIs, integrations, tests and release evidence
Ticket	SALDEV-1500

Executive decision: Make GitHub Enterprise Cloud the canonical source repository, keep Jira as the canonical work and approval record, and use a Salesforce-aware release platform on top. Prefer Gearset for a new implementation; use Copado when an existing enterprise investment makes it the lower-risk choice.

Approval requested

- Approve the target operating model and the principle that every production change starts with a Jira ticket and ends with evidence linked back to that ticket.
- Authorize procurement or confirmation of GitHub Enterprise Cloud and the selected Salesforce DevOps platform for the required users and service accounts.
- Create the implementation epic and stories in Section 17, then pilot the process with one low-risk Salesforce change before scaling.

Contents

- 1. Executive summary and ticket outcomes
- 2. Scope, exclusions and operating principles
- 3. Tool recommendation and decision rationale
- 4. Target operating architecture
- 5. Repository portfolio and Salesforce folder structure
- 6. Version-control and branching standard
- 7. End-to-end delivery and deployment process
- 8. Jira workflow, gates and automation
- 9. CI/CD quality gates by stage
- 10. Environment, deployment and release controls
- 11. Security, access and secrets
- 12. Rollback, hotfix and drift management
- 13. Roles and accountability
- 14. Definition of Ready and Definition of Done
- 15. Confluence publication structure
- 16. Implementation roadmap and success measures
- 17. Jira-ready implementation stories
- 18. Acceptance-criteria traceability and ready-to-paste response
- Appendix A. Repository tree
- Appendix B. Sources and assumptions

How to use this document

The opening sections establish the decision. Sections 5 through 14 are the operating standard. Sections 15 through 17 are the implementation package. The appendices contain a copy-ready repository tree and the source basis. This document is designed to be imported or copied into the GTM Systems Confluence space without changing its control logic.

1. Executive summary and ticket outcomes

Flywire's parallel GTM roadmap creates a version-control problem and an operating-model problem. Code, declarative Salesforce metadata, CPQ/OmniStudio configuration, integration assets, tests and runbooks must move through one traceable process even when different teams work at the same time. SALDEV-1500 is therefore not only a repository setup task; it establishes the rules connecting Jira, Git, automated validation, environment promotion and production evidence.

Decision

- Repository: GitHub Enterprise Cloud, with private repositories, enterprise identity controls, protected default branches, CODEOWNERS, required pull-request reviews and required status checks.
- Work management: Jira remains the authoritative backlog, scope, acceptance, approval and release-evidence record. Git commits, branches, pull requests and deployment jobs must carry the Jira key.
- Salesforce delivery: Gearset is the preferred greenfield release layer because it provides Salesforce-aware comparisons, dependency analysis, source-control integration, CI jobs and Jira integration. Copado is the approved alternative when Flywire already has the licenses, platform ownership and skills to operate it effectively.
- Cross-GTM CI: GitHub Actions provides repository-level checks and reusable automation for integration code, scripts, documentation and non-Salesforce components. The Salesforce deployment platform owns metadata-aware promotion and validation.

Evidence basis: GitHub documents protected branches and deployment environments with required checks/reviewers [S1-S3]; Gearset documents end-to-end CI and Git/Jira integrations [S4-S6].

SALDEV-1500 outcomes

Ticket requirement	Deliverable in this operating model	Outcome
Recommend a repository/version-control tool	Section 3 decision and decision matrix	Addressed
Define repository and folder structure	Sections 5-6 and Appendix A	Addressed
Document deployment and CI/CD process	Sections 7, 9 and 10	Addressed
Document Jira ticket movement	Section 8 workflow and transition evidence	Addressed
Create implementation stories	Section 17 copy-ready Jira stories	Addressed
Publish in GTM Systems Confluence	Section 15 publication tree and owner actions	Implementation story required

Decision boundaries

Not authorized by this document: No repository, Jira workflow, connected app, Salesforce environment, CI/CD pipeline or production deployment is created or changed by this planning deliverable. Each implementation action requires its approved Jira story and named owner.

2. Scope, exclusions and operating principles

In scope

- Salesforce Apex, Lightning Web Components, Flows, objects, fields, layouts, permissions, OmniStudio assets, CPQ configuration represented in supported source formats, Experience Cloud assets and deployment manifests.
- Integration code and configuration for APIs, middleware, eventing and downstream GTM systems, provided secrets and environment values are externalized.
- Automated tests, static analysis rules, test data definitions, release scripts, runbooks, architecture decisions and deployment evidence.
- Business-as-usual changes, platform-unification work and new quote-to-cash capabilities, all using the same control path with risk-scaled gates.

Out of scope for the initial implementation

- Replacing Jira, Confluence or an enterprise identity provider.
- Placing production data, credentials, tokens, certificates, unmasked exports or customer documents in Git.
- Automating high-risk production changes before the pilot proves the validation, approval and rollback controls.
- Forcing all GTM systems into one repository when they have different release cadences, owners or access boundaries.

Non-negotiable principles

Principle	Operating rule
One traceable unit of change	Jira key → branch → commit → pull request → validation → deployment → evidence.
Git is the source of truth	No unmanaged production-intent metadata exists only in an org or on a workstation.
No direct production changes	Emergency changes use the hotfix path and are reconciled to Git immediately.
Separation of duties	The author cannot be the sole reviewer and production approver.

Principle	Operating rule
Small, reversible changes	Prefer focused tickets and deployable increments; isolate destructive changes.
Evidence before status	A Jira transition represents completed evidence, not an expectation.
Least privilege and no secrets in Git	Use enterprise identity, service accounts, protected secrets and short-lived credentials.
Automation with a human production gate	Automate repeatable checks; retain explicit approval for production.

3. Tool recommendation and decision rationale

Name confirmation: The meeting transcript says “GitSense.” This document interprets that as Gearset, the Salesforce DevOps platform. If a different product was intended, confirm the product name and repeat the procurement comparison before commitment.

Recommended stack: GitHub Enterprise Cloud + Jira + Gearset, with GitHub Actions for repository-native checks. If Flywire already operates Copado at enterprise scale, use GitHub Enterprise Cloud + Jira + Copado while preserving the same controls and evidence model.

Why GitHub is the repository decision

The repository tool must serve Salesforce and adjacent GTM engineering, not only metadata deployment. GitHub Enterprise Cloud provides the shared Git system of record, review controls and automation surface. GitHub rulesets and protected branches can require pull requests and successful checks, while deployment environments can require reviewers and restrict deployment branches. These are foundational controls regardless of which Salesforce-aware release product is selected. [S1-S3]

Why the Salesforce release layer is separate

Salesforce changes have dependency, profile/permission, destructive-change and environment-drift behavior that generic Git automation does not fully model. A Salesforce-aware platform reduces manual packaging and gives admins a guided promotion path. Gearset documents CI jobs that compare version control with Salesforce orgs, build packages, analyze dependencies, run tests and retain deployment reports; it also documents GitHub and Jira integrations. [S4-S6]

Decision matrix

Option	Position	Strength	Primary watch-out
GitHub Enterprise + Gearset	Preferred greenfield	Broad Git governance plus Salesforce-aware CI/CD, dependency analysis, Jira integration and admin-friendly operations.	Confirm enterprise/security review, data residency, license tiers and service-account model.
GitHub Enterprise + Copado	Approved alternative	Strong Salesforce-native promotion model, user-story/branch associations and pipeline controls; good fit when Copado capability already exists.	Avoid duplicate Jira/Copado story administration; confirm Source Format feature limitations and operating ownership. [S7-S8]
GitHub Enterprise + Actions + sf CLI	Engineering-led fallback	Maximum code transparency and cross-platform reuse; no additional Salesforce deployment vendor required.	Higher build/maintenance burden and a steeper path for declarative admins; team owns dependency handling, UX and audit reporting.
Salesforce DevOps Center	Pilot/limited fit	Native Salesforce option with GitHub connection and work-item concepts.	Assess fit for complex GTM integrations, CPQ/OmniStudio scope, cross-system releases and enterprise reporting before selection.

Procurement decision gates

1. Confirm whether Copado licenses, administrators, pipeline configuration and support ownership already exist at Flywire.
2. Run the same pilot scenario in Gearset and Copado if there is no decisive existing investment: retrieve a representative Salesforce change, create a ticket-linked branch, validate a pull request, promote to a test org and export audit evidence.
3. Score security architecture, SSO/SCIM, role segregation, audit retention, Salesforce/CPQ/OmniStudio support, Jira linkage, API/export access, service-account licensing, support model and total three-year cost.
4. Do not choose a product solely because it can deploy metadata; the winning tool must support the operating controls in this document.

4. Target operating architecture

One governed path from demand to production

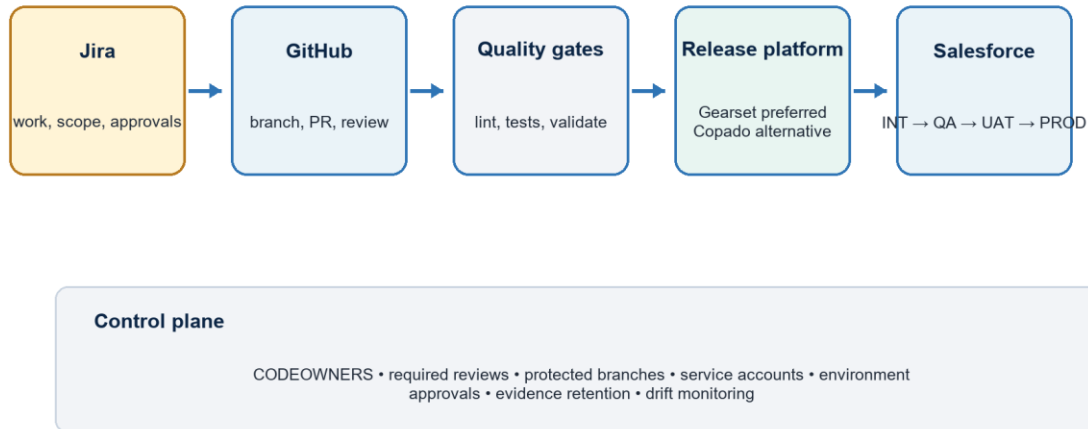


Figure 1. Target control path. Git and Jira remain authoritative even when the Salesforce release platform performs promotion.

System-of-record boundaries

System	Authoritative for	Must link to
Jira	Business scope, acceptance criteria, ownership, risk, approvals, status and release evidence	Branch, PR, validation and deployment records
GitHub	Versioned code/configuration, branch history, reviews, tags and repository automation	Jira ticket and deployment platform
Gearset or Copado	Salesforce comparison, package construction, validation, promotion and deployment audit	Git commit/PR and Jira ticket
Confluence	Published standards, runbooks, architecture decisions, training and release calendar	Jira implementation stories and repository source
Salesforce orgs	Runtime state only; not the long-term version history	Deployed commit/tag and release record

Minimum integration contract

- Branch format begins with the Jira key; pull request title begins with the Jira key; commits include the Jira key.
- The Jira development panel or equivalent links the branch, PR and build/deployment events.

- A deployment record identifies source commit/tag, target environment, component scope, test level/results, approver, operator, timestamps and outcome.
- Production credentials are available only to the protected production job/environment, after approval, and never to pull-request jobs.
- The evidence link is written back to Jira before the ticket moves from Deployed to Done.

5. Repository portfolio and Salesforce folder structure

Portfolio decision: domain-aligned repositories

Use a small set of repositories aligned to deployable systems and access boundaries. Do not create one repository per ticket and do not place every GTM technology in one monorepo by default. Start with the portfolio below and add a repository only when ownership, release cadence, security boundary or build tooling is materially different.

Repository	Contents	Primary owner
gtm-salesforce-core	Unified Salesforce metadata, Apex/LWC, supported CPQ and OmniStudio source, manifests and tests	GTM Salesforce engineering
gtm-integrations	API/middleware services, schemas, mappings, contract tests and deployment code	Integration engineering
gtm-devops-templates	Reusable workflows, repository policies, scripts and quality configurations	Platform & DevOps
gtm-architecture-and-runbooks	ADRs, diagrams, operating standards, release/rollback runbooks and Confluence source	Architecture + release management
gtm-data-config	Non-secret, deployable configuration and synthetic seed definitions that cannot live naturally with system code	System owners; create only when needed

Salesforce repository standard

Path	Purpose and rule
.github/	CODEOWNERS, PR template, issue forms, workflows and dependency/update policy.
force-app/main/default/	Salesforce DX source format. Organize deployable metadata by native metadata folders.
packages/<domain>/	Optional package directories for bounded capabilities such as quoting, pricing, contracting or shared platform; introduce only after dependency analysis.
manifest/	Canonical manifests, environment-safe destructive manifests and retrieval/deployment scopes.
config/	Scratch-org/project definitions and non-secret tool configuration.
scripts/ci/	Deterministic validation, delta/package generation, analysis and test orchestration.

Path	Purpose and rule
scripts/release/	Release packaging, evidence capture, tag verification and approved operational helpers.
tests/	Integration, UI, contract and test-data definitions not colocated with source.
data/seed/	Synthetic or masked seed plans only; never production exports or personal data.
docs/	Architecture decisions, component maps, runbooks, release notes and manual deployment steps.
Root files	README.md, CONTRIBUTING.md, SECURITY.md, CODEOWNERS reference, sfdx-project.json, package.json and ignore files.

Repository content rules

- Commit text-based source whenever the platform supports it. Use Git LFS only for necessary large binaries and after platform/security approval.
- Exclude credentials, tokens, certificates, auth URLs, customer data, production exports, generated build outputs, local CLI state and machine-specific files through .gitignore and secret scanning.
- Keep environment-specific IDs, endpoints and secrets outside deployable source where possible; use named configuration, protected variables or post-deployment tasks with documented ownership.
- Every manual step must live in a versioned runbook and be referenced by the release record; undocumented manual setup is a release blocker.

6. Version-control and branching standard

Branch model: Use short-lived ticket branches plus controlled environment branches for the Salesforce promotion path. Keep main aligned to production intent and tag every production release.

Branch	Purpose	Control
main	Production intent and release history	Protected; no direct push; release manager merge; tag after successful production deployment.
uat	Approved UAT candidate	Protected; promotion PR only; successful UAT validation required.
integration	Shared integration baseline	Protected; feature PRs target this branch after review and validation.
SALDEV-<key>-<slug>	Short-lived feature/fix branch	Create from the current approved base; delete after promotion; rebase/merge per repository policy.

Branch	Purpose	Control
release/<yyyy.mm>	Optional coordinated release train	Create only when a freeze or parallel release is required; not a permanent default.
hotfix/SALDEV-<key>-<slug>	Emergency correction from main	Requires incident/change reference, expedited review and mandatory back-promotion.

Naming and traceability

- Branch: SALDEV-1523-price-ramp-validation
- Commit: SALDEV-1523 Add validation for ramped quote groups
- Pull request: SALDEV-1523 — Validate stepped pricing dates
- Production tag: salesforce-v2026.09.15.1
- Release record: includes the immutable commit SHA and production deployment job ID.

Default branch/ruleset controls

- Require pull requests, two approvals for production-path merges and at least one CODEOWNER approval for owned paths.
- Dismiss stale approvals when code changes; require conversation resolution and successful status checks.
- Block force pushes and branch deletion; restrict bypass to a very small audited break-glass group.
- Require signed commits if enterprise policy supports the user/tooling model; otherwise prioritize enforced reviews and immutable audit logs.
- Use merge commits or squash merges consistently. Recommended: squash feature PRs for one-ticket/one-change traceability, preserve promotion merges as configured by the release platform.

GitHub documents protected branches and enterprise rulesets that can require PRs, reviews and status checks [S2-S3].

7. End-to-end delivery and deployment process

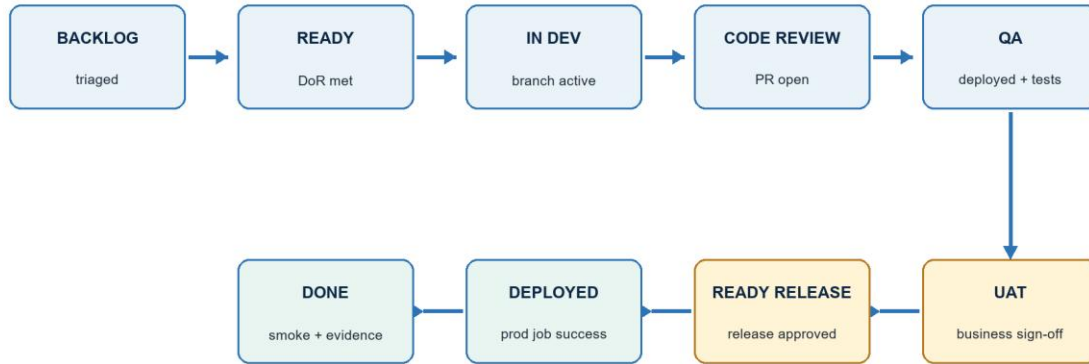
Stage	Accountable role	Required evidence	Exit status
1. Refine	Product owner + architect	Scope, acceptance criteria, metadata/data/API impact, risk, dependencies, test personas, rollback outline	Ready for Dev
2. Branch	Developer/admin	Ticket-linked branch from approved base; environment synchronized; no unrelated components	In Development
3. Build	Developer/admin	Source captured in DX/source format; local checks; manual steps documented; no secrets	In Development
4. Pull request	Author	Focused diff, risk, test evidence, component list, rollback, screenshots where relevant	Code Review
5. Validate	CI + reviewer	Static analysis, unit tests, metadata validation/check-only and dependency review	Ready for QA
6. Integrate	Release platform	Promote approved commit to INT/QA; capture job ID and scope	In QA
7. Test	QA	Regression, permission/persona, integration and negative-path evidence	Ready for UAT
8. UAT	Business owner	Acceptance criteria executed; named approval recorded; defects linked	Ready for Release
9. Release	Release manager	Change window, production validation, approvals, runbook, backup/rollback readiness	Scheduled
10. Deploy	Approved operator/service	Protected production job; immutable source commit; no scope substitution	Deployed
11. Verify	Release + business	Smoke tests, monitoring, business validation, deployment evidence and known issues	Done

Change classes

Class	Definition	Required path
Standard	Repeatable, low-risk, no destructive/security/integration impact	Normal PR + automated validation + QA; predefined release approval.
Normal	Most functional/configuration/code changes	Full process including UAT and release-manager approval.
High risk	Permissions, authentication, data model deletion, critical integration, billing/quote logic or broad automation	Architect/security review, expanded regression, rehearsed rollback and explicit business approval.
Emergency	Active incident or urgent regulatory/security fix	Hotfix path, expedited independent review, production approval and retrospective/back-promotion within one business day.

8. Jira workflow, gates and automation

Jira workflow and release evidence



Rejected validation or approval returns the ticket to IN DEV with the failure evidence attached.

Figure 2. Proposed Jira lifecycle. Status changes are evidence gates, not manual progress labels.

Status	Entry condition	Mandatory evidence	Transition owner
Backlog	Request accepted for triage	Problem/value, requester, affected system	Product owner
Ready for Dev	Definition of Ready is complete	Acceptance criteria, design/risk, dependencies, estimate, test and rollback approach	Product owner + architect
In Development	Ticket branch exists	Branch link and assignee	Developer/admin
Code Review	Pull request opened	PR link, component list, test evidence, risk/rollback	Reviewer/CODE OWNER
Ready for QA	PR approved and validation passed	Review approvals and successful required checks	Dev lead
In QA	Approved commit deployed to QA	Deployment job ID, commit SHA, scope and test plan	QA lead
Ready for UAT	QA exit criteria passed	QA results and defects disposition	QA lead
In UAT	UAT environment ready	UAT version/commit, test cases and named testers	Business owner
Ready for Release	UAT approved	UAT sign-off, release notes, risk class and rollback	Business owner + architect
Scheduled	Production approval and change window confirmed	Release record, approver, runbook and production validation	Release manager
Deployed	Production job succeeded	Job ID, commit/tag, scope, tests and operator	Release manager
Done	Smoke/business checks passed	Post-deploy evidence, incidents/known issues and final release link	Product owner

Transition and approval controls

- Do not allow unrestricted Any Status transitions into approval-controlled statuses. Atlassian notes that permissive transitions can bypass approval controls. [S9-S10]
- If Jira Premium approval steps are available, use distinct approval fields for UAT and Production Release. If not, use validators/conditions plus named approver fields and immutable approval comments.
- A failed validation, rejected review, QA defect or declined UAT returns the ticket to In Development with the failure/reason linked.
- Automations may advance a ticket only from trusted events (for example, an approved PR or successful deployment webhook) and must never mark Done on deployment success alone.
- Subtasks may track build, testing or documentation, but the parent ticket owns the end-to-end release status and evidence.

Required Jira fields

Field	Purpose
Affected system/repository	Controls owner and workflow routing
Change class	Standard / Normal / High risk / Emergency
Target release	Release train or approved production window
Component/dependency summary	Metadata, code, APIs, data/config and downstream dependencies
Test strategy	Automated tests, regression scope, personas and data setup
Rollback/forward-fix plan	Action, decision point and owner
UAT approver + approval	Named business approval evidence
Release approver + approval	Named production approval evidence
PR/build/deployment links	Automated where supported
Production commit/tag and job ID	Immutable release evidence

9. CI/CD quality gates by stage

Gate	Checks	Owner	Pass rule
Commit/local	Formatting, lint, focused unit tests, secret pre-check, metadata source sanity	Author	Fail blocks PR readiness
Pull request	Jira key, allowed paths, CODEOWNERS, secret scan, static analysis, Apex/LWC tests, metadata validation/check-only, dependency/package review	CI + reviewer	All required checks pass
Integration	Deploy approved commit, integration tests, scheduled Apex conflicts, API/contract checks	Release platform	Job and test evidence retained
QA	Regression, permission/persona coverage, negative paths, CPQ/OmniStudio/integration scenarios	QA	No open release blocker
UAT	Business acceptance criteria against immutable commit; training/release notes checked	Business owner	Named approval
Production validation	Same package/commit, target-org validation, prescribed Apex tests, manual-step review	Release manager	Validation within approved window
Production	Environment approval, protected credential, approved package only, job logging	Release manager/operator	Successful deployment
Post-production	Smoke tests, monitoring, key persona check, evidence write-back	Release + business	Done criteria met

Minimum Salesforce pull-request checks

- Validate the exact changed package against the target integration or validation org using Salesforce CLI or the selected platform; do not treat a source parse as a deployability check. [S11]
- Run impacted Apex tests at minimum; require broader regression for shared triggers/frameworks, security changes, critical revenue logic and production validation.
- Run LWC Jest tests and lint where relevant; run static analysis with an approved ruleset and documented suppressions.
- Check profiles and permission sets for unintended removals or broad access. Treat permissions as whole-file changes where platform behavior requires it.
- Detect destructive changes, renamed metadata, missing dependencies, unsupported metadata and environment-specific references; route any destructive change to high-risk approval.
- Store validation job IDs, component counts, test counts/results and omissions. A partial or blocked validation is not a pass.

Evidence schema

Evidence category	Required content
Identity	Jira key, repository, branch, PR, commit SHA/tag
Scope	Added/changed/deleted components; package/manifest; manual steps; explicit exclusions
Target	Named environment/org alias and organization identifier where policy permits
Validation	Job ID, status, timestamp, test level, tests run/passed/failed and static-analysis outcome
Approval	Reviewer, UAT approver, release approver and approval timestamp
Deployment	Production job ID, operator/service identity, start/end time and result
Verification	Smoke checks, monitoring, incidents, rollback decision and final sign-off

10. Environment, deployment and release controls

Environment	Purpose	Owner	Control
Developer / scratch	Isolated build and focused tests	Developer/admin	No production credentials; ticket branch only
Integration	Shared dependency and integration validation	Dev lead	Automatic after approved merge where safe
QA	Functional and regression testing	QA lead	Approved commit; controlled test data
UAT	Business acceptance and release candidate	Business owner	Immutable candidate; no untracked fixes
Production	Live service	Release manager + system owner	Protected job, explicit approval, least privilege, post-deploy verification

Release readiness checklist

- All included Jira tickets are Ready for Release and identify the same immutable commit/tag.
- Production validation passed for the exact package and target, with current job evidence.
- Dependencies, manual steps, data/configuration tasks, integration sequencing and expected downtime are documented.
- No unresolved blocker or unapproved scope change exists; exceptions have named acceptance and expiry.
- Rollback/forward-fix decision points, owners, scripts and backups are ready and rehearsed for high-risk changes.

- Communications, monitoring and business smoke-test owners are scheduled.
- The production operator is different from the sole code author and has the correct least-privilege access.

Production deployment rules

- Production deployment runs only from the approved release commit/tag; never from a workstation's uncommitted state or an arbitrary org comparison.
- No component may be added after validation without returning to review and revalidation.
- Use the platform's validation/quick-deploy capability only when the validated artifact, tests and validity window meet Salesforce and company policy.
- Manual steps require two-person verification for high-risk actions and a timestamped execution record.
- After deployment, create the release tag and release notes only when the actual production state is confirmed; if tagging occurs before execution, the release record must distinguish planned from deployed.

11. Security, access and secrets

Control area	Required standard
Human access	Enterprise SSO, MFA, group-based teams, least privilege and periodic recertification.
Service accounts	Dedicated non-human identities; no shared personal credentials; named owner and rotation plan.
Secrets	Repository/environment secret store or approved enterprise vault; never Git, Jira comments, build logs or documents.
Production gate	Protected deployment environment with required reviewer(s), branch restriction and no self-approval where supported. [S1]
Actions/workflows	Allow approved actions only; pin third-party actions to immutable versions/SHAs per enterprise policy. [S3]
Audit	Retain Git, Jira and deployment-platform audit records according to Flywire policy; export when vendor retention is insufficient.
Data	Use synthetic or masked non-production data; do not commit Salesforce exports or personal/customer data.
Break glass	Small named group, time-bound access, mandatory reason, immediate notification and retrospective review.

Access model

Role	Access boundary
Contributor	Create ticket branches and PRs; no protected-branch push; no production deploy.
Reviewer / CODEOWNER	Review owned paths; cannot replace release approval.
Release operator	Run approved promotion/deployment jobs; cannot bypass scope/review.
Release approver	Approve production environment/job; ideally not the author or operator.
Repository admin	Manage policy and teams; no routine bypass; changes are audited.
Security auditor	Read audit/configuration evidence; no deployment mutation.

12. Rollback, hotfix and drift management

Rollback strategy

Salesforce rollback is change-specific. The default is a controlled forward fix or a revert commit deployed through the same pipeline; destructive metadata and data changes may require restoration or compensating actions. Every ticket must therefore state the actual rollback boundary rather than using a generic sentence.

Change type	Rollback expectation
Apex/LWC/Flow/config	Revert the approved commit or deploy the prior known-good version; validate dependencies and activation state.
Permissions	Restore the prior permission source; verify user/persona access and removals before/after deployment.
Destructive metadata	Require pre-change backup and explicit restoration/forward-fix runbook; deletion may not be trivially reversible.
Data/schema migration	Back up affected data, define transformation reversal or compensating migration, and identify ownership.
Integration/API	Feature flag, route/version rollback, credential/config reversal and partner communication as applicable.

Emergency hotfix flow

1. Open or link an incident and emergency Jira ticket; record impact, urgency and approver.
2. Create hotfix/SALDEV-`<key>-<slug>` from main and make the smallest safe change.
3. Run focused automated validation and obtain independent technical review; record any skipped gates and risk acceptance.

4. Deploy through the protected production path with emergency approval and capture the job evidence.
5. Verify service recovery, then merge/back-promote the hotfix through UAT/integration branches so future releases retain it.
6. Complete a retrospective and close any missing test, documentation or automation debt within one business day or the agreed incident SLA.

Drift management

- Run scheduled source-to-org comparison/change monitoring for production and key sandboxes; classify differences as authorized, expected platform behavior or unmanaged drift.
- Block routine direct production changes. If an emergency change occurs, retrieve/reconcile it into Git immediately and link the incident/ticket.
- Refresh the production baseline only through a reviewed pull request. Never overwrite Git blindly from a broad org retrieve.
- Report drift age, unowned differences and repeated bypass patterns as operating metrics.

13. Roles and accountability

Activity	Product owner	Architect/dev lead	Developer/ad min	QA	Business/U AT	Release mgr
Define scope/acceptance	A	C	R	C	C	I
Design/risk review	C	A	R	C	C	C
Build and local test	I	C	A/R	C	I	I
Code/config review	I	A	R	C	I	C
QA/regression	I	C	C	A/R	I	I
UAT approval	C	C	I	C	A/R	I
Release readiness	C	C	C	C	C	A/R
Production approval	C	C	I	I	A	R
Production deployment	I	C	I	I	I	A/R
Post-deploy verification	A	C	C	R	R	R

R = Responsible, A = Accountable, C = Consulted, I = Informed. Security and platform administration are mandatory consultees/approvers for changes in their control domains.

Operating ceremonies

Cadence	Purpose
Weekly intake/refinement	Prioritize requests; establish DoR, dependencies, change class and release target.
Daily pipeline review	Review failed validations, blocked promotions, drift alerts and aging PRs.
Release readiness review	Confirm evidence, package scope, approvals, runbook and rollback before production window.
Post-release review	Confirm business outcome, incidents, metrics, evidence completeness and follow-up actions.
Monthly control review	Review access, bypasses, drift, failed change rate, tool health and process exceptions.

14. Definition of Ready and Definition of Done

Definition of Ready

- Business outcome, affected personas and measurable acceptance criteria are clear.
- Affected system/repository, component types, dependencies and integration/data impacts are identified.
- Change class, architecture/security review needs and release target are set.
- Test strategy covers automated tests, QA regression, UAT personas and required test data.
- Rollback/forward-fix approach and manual steps are plausible and have owners.
- Required environments, access and upstream prerequisites are available or explicitly tracked.
- Story is sized and small enough to review, validate and reverse independently where practical.

Definition of Done

- Approved source is merged and the ticket, PR, commit and deployment records are linked.
- Required automated checks, target-org validation, QA and UAT are complete with exact evidence.
- Production deployment succeeded from the approved immutable commit/tag with no unapproved scope changes.
- Post-deployment smoke and business checks passed; incidents/known issues are linked and owned.
- Documentation, runbooks, release notes and monitoring updates are published where required.

- Manual changes are reconciled into Git; temporary access/flags are removed or have an expiry story.
- The Jira ticket contains production job ID, component/test evidence, approvers, outcome and final release link.

Closure rule: Deployment success alone is not Done. Done requires verified behavior, complete evidence and reconciliation to the source of truth.

15. Confluence publication structure

Publish this material under the GTM Systems space as a controlled page tree. The operating model should be readable by leadership, while detailed runbooks remain versioned and owned. Add page owners and next-review dates; use the repository as the source for technical artifacts where practical.

Page	Purpose
GTM Systems Engineering	Landing page, ownership, links to standards and current release calendar
↳ DevOps & Release Management	Operating-model home and policy status
↳ SALDEV-1500 — DevOps Operating Model	Executive decision, principles, architecture and RACI
↳ Repository & Branching Standard	Repo portfolio, folder tree, naming, CODEOWNERS and branch policy
↳ Jira Delivery Workflow	Statuses, fields, transition gates, approvals and automation
↳ CI/CD Quality Gates	Checks, test strategy, evidence schema and exception process
↳ Deployment & Rollback Runbook	Environment promotion, production checklist, rollback and hotfix
↳ Tool Administration	GitHub and Gearset/Copado configuration, access, service accounts and audit
↳ Training & Onboarding	Contributor, reviewer, QA and release-manager guides
↳ Metrics & Continuous Improvement	DORA-style metrics, drift, bypasses, failures and monthly actions

Publication controls

- Page owner: Head of GTM Systems or delegated Platform & DevOps lead.
- Review cadence: quarterly and after any tool, environment, workflow or production-control change.
- Changes to mandatory controls require a Jira change, architecture/release approval and version-history note.

- Runbook commands and configuration examples must be tested in a non-production environment before publication as operational instructions.

16. Implementation roadmap and success measures

Wave	Indicative timing	Scope	Exit
Wave 0 — Decide	Weeks 0-2	Confirm product name, existing contracts, security/procurement, owners and pilot scope	Signed decision and funded implementation epic
Wave 1 — Foundation	Weeks 2-5	GitHub organization/teams, repository template, baseline metadata, access/secrets and Jira fields/workflow	Controlled repository and ready backlog
Wave 2 — Automate	Weeks 5-8	PR checks, Salesforce validation, release platform pipeline, evidence links and runbooks	End-to-end non-production pipeline
Wave 3 — Pilot	Weeks 8-10	One low-risk change through INT, QA, UAT and protected production with rollback rehearsal	Pilot acceptance and documented lessons
Wave 4 — Scale	Weeks 10-14	Training, repository migration, change freeze/cutover plan, metrics and control review	BAU operating model adopted

90-day success measures

Measure	Initial target / method
Production traceability	100% of pilot-scope production changes linked Jira → PR → commit/tag → deployment job.
Protected-path adoption	100% of main/production-path merges through approved PRs; zero routine direct pushes.
Validation evidence	100% of release candidates have target-org validation status, job ID, test counts and explicit omissions.
Release quality	Track failed change rate and rollback/forward-fix events; set the steady-state target after the pilot baseline.
Lead time	Measure Ready for Dev to Deployed and identify queue time at review, QA, UAT and release gates.
Drift	All detected production drift triaged within two business days; unauthorized drift reconciled through Jira/Git.
Evidence completeness	At least 95% of Done tickets pass monthly evidence audit in month one; 100% by month three.

Pilot exit criteria

- A low-risk ticket completes the full workflow without direct production changes or manual source substitution.

- Required checks block an intentionally failing PR and allow a corrected PR.
- QA/UAT/production jobs all reference the same approved source lineage.
- The team executes the rollback rehearsal in non-production and validates evidence retrieval.
- Users representing developer/admin, reviewer, QA, business approver and release manager confirm the process is usable.
- Open gaps have owners, severity and due dates; no critical control gap remains before scale-out.

17. Jira-ready implementation stories

Backlog usage: Create these stories under a new SALDEV implementation epic linked to SALDEV-1500. Estimates are indicative planning placeholders and must be re-estimated by the delivery team after tool and environment discovery.

SALDEV-1500-A01 — Approve repository and Salesforce DevOps toolchain

Field	Value
Jira issue type	Story
Suggested owner	Head of GTM Systems + Procurement
Dependency	None
Indicative estimate	5 points
Delivery wave	Wave 0

User story

As the GTM Systems leadership team, we want an approved source-control and release-tool decision so that implementation proceeds with clear ownership, security approval and funding.

Acceptance criteria

- GitHub Enterprise Cloud is confirmed as the repository standard or an exception is formally documented.
- The transcript term “GitSense” is confirmed as Gearset or corrected to the intended product.
- Existing Copado contracts, licenses, administrators and configured pipelines are inventoried.
- Gearset and Copado are scored against the procurement gates in Section 3 using the same pilot scenario.
- Security, privacy, data residency, SSO/SCIM, audit retention and vendor risk reviews are complete.
- Decision record names the selected platform, accountable owner, license quantities, service-account needs and three-year cost assumptions.

SALDEV-1500-A02 — Provision GitHub Enterprise organization, teams and base controls

Field	Value
Jira issue type	Story
Suggested owner	Platform & DevOps
Dependency	A01
Indicative estimate	8 points
Delivery wave	Wave 1

User story

As Platform & DevOps, we want the GitHub organization and access model configured so that GTM source is private, governed and auditable.

Acceptance criteria

- Enterprise organization, billing owner and support contacts are confirmed.
- SSO/MFA and team-based access are enabled; contributor, reviewer, release and admin groups are mapped.
- Default repository visibility is private and external collaboration policy is configured.
- Enterprise/organization rulesets protect main and designated promotion branches from direct push, force push and deletion.
- Required PR reviews, CODEOWNER review, status checks, conversation resolution and bypass audit are configured.
- Audit-log ownership and retention/export procedure are documented and tested.

SALDEV-1500-A03 — Create GTM repository template and Salesforce repository

Field	Value
Jira issue type	Story
Suggested owner	Platform & DevOps + Salesforce lead
Dependency	A02
Indicative estimate	8 points
Delivery wave	Wave 1

User story

As a GTM engineer, I want a standard repository template and Salesforce repository so that every change begins with consistent structure, controls and documentation.

Acceptance criteria

- gtm-devops-templates and gtm-salesforce-core repositories are created using the structure in Section 5 and Appendix A.
- README, CONTRIBUTING, SECURITY, CODEOWNERS, PR template, ignore files and ownership metadata are present.
- Salesforce DX project configuration, force-app path, manifests, scripts and test directories are valid.
- Repository contains no credentials, auth URLs, production data, generated artifacts or local CLI state.
- Template contains a sample ticket-linked branch and PR that passes policy checks.
- Repository creation procedure is documented for future GTM systems.

SALDEV-1500-A04 — Create reviewed Salesforce production baseline

Field	Value
Jira issue type	Story
Suggested owner	Salesforce lead + Architect
Dependency	A03
Indicative estimate	13 points
Delivery wave	Wave 1

User story

As the Salesforce engineering team, we want a reviewed source baseline of the current production-intent metadata so that Git becomes the controlled source of truth without hiding existing drift or omissions.

Acceptance criteria

- Approved metadata scope and explicit exclusions are documented before retrieval.
- Baseline is retrieved using a dedicated read-only/least-privilege process and committed to a temporary review branch.
- Unsupported, missing or partial metadata and source-format conversion issues are listed with owners.
- Sensitive/environment-specific values and generated files are excluded or externalized.
- Architect and system owner approve the baseline PR after component-count and drift review.
- No deployment or production mutation occurs as part of the baseline creation.

SALDEV-1500-A05 — Configure service identities, secrets and environment access

Field	Value
Jira issue type	Story
Suggested owner	Security + Platform Operations
Dependency	A01-A03
Indicative estimate	8 points
Delivery wave	Wave 1

User story

As Security and Platform Operations, we want controlled non-human identities and secret storage so that CI/CD can validate and deploy without shared personal credentials.

Acceptance criteria

- One named service identity per required trust boundary is documented with owner, purpose and expiry/rotation process.
- Non-production and production credentials are separated; pull-request jobs cannot access production credentials.
- Secrets are stored only in the approved vault/repository environment store and are masked from logs.
- Production environment requires named approval and prevents self-approval where supported.
- Least-privilege Salesforce and repository permissions are verified through an access test.
- Break-glass access, alerting and quarterly recertification procedures are documented.

SALDEV-1500-A06 — Implement pull-request quality and Salesforce validation gates

Field	Value
Jira issue type	Story
Suggested owner	Platform & DevOps + Salesforce lead
Dependency	A03-A05
Indicative estimate	13 points
Delivery wave	Wave 2

User story

As a reviewer, I want automated pull-request checks so that unsafe or untraceable changes cannot merge into the promotion path.

Acceptance criteria

- Workflow validates Jira key, allowed branch naming, changed paths and required PR fields.
- Secret scanning, lint/static analysis, relevant LWC tests and Apex test strategy are configured.
- Exact changed Salesforce scope is validated/check-only against the approved target org or platform job.
- Required checks report job ID, component counts, tests run/passed/failed, warnings and explicit omissions.
- An intentionally failing fixture blocks merge and a corrected fixture passes.
- Workflow actions/dependencies are approved, pinned and owned; troubleshooting runbook is published.

SALDEV-1500-A07 — Configure Salesforce release pipeline across INT, QA, UAT and Production

Field	Value
Jira issue type	Story
Suggested owner	Release manager + Salesforce lead
Dependency	A01, A04-A06
Indicative estimate	13 points
Delivery wave	Wave 2

User story

As a release manager, I want a controlled Salesforce promotion pipeline so that one approved source lineage moves through every environment with auditable gates.

Acceptance criteria

- Selected Gearset or Copado platform is connected to GitHub and the named Salesforce environments using approved service identities.
- Branch/environment mapping, metadata filters, permission behavior, test levels and deployment ownership are documented.
- Pull-request validation and non-production promotion jobs retain immutable source, target, scope, test and outcome evidence.
- UAT and production steps require named approvals; production cannot run from unapproved branches or ad hoc source.
- Destructive changes and manual deployment tasks use separate high-risk controls.
- A dry-run/non-production test proves promotion, failure handling, evidence export and rollback rehearsal.

SALDEV-1500-A08 — Implement Jira delivery workflow, fields and automation

Field	Value
Jira issue type	Story
Suggested owner	Jira admin + Delivery lead
Dependency	A01 and workflow approval
Indicative estimate	13 points
Delivery wave	Wave 1-2

User story

As a delivery team, we want Jira statuses and evidence gates aligned with CI/CD so that ticket status accurately reflects delivery state.

Acceptance criteria

- Statuses, transitions and rework paths match Section 8 and are reviewed with product, QA, architecture and release roles.
- Required fields for change class, system, dependencies, test, rollback, UAT, release approval and deployment evidence are configured.
- Approval-controlled statuses cannot be reached through unrestricted transitions.
- Trusted branch/PR/build/deployment events update links or status without marking Done prematurely.
- Rejected review, failed validation, QA defect and declined UAT return to In Development with evidence.
- A sandbox Jira project or safe test issue demonstrates each transition, permission and automation path.

SALDEV-1500-A09 — Publish deployment, rollback, hotfix and drift runbooks

Field	Value
Jira issue type	Story
Suggested owner	Release manager + Architect
Dependency	A07-A08
Indicative estimate	8 points
Delivery wave	Wave 2

User story

As an on-call release team, we want tested runbooks so that normal, emergency and recovery actions are repeatable and auditable.

Acceptance criteria

- Normal release checklist covers readiness, production validation, approval, execution, smoke test and evidence write-back.
- Rollback patterns address code/config, permissions, destructive metadata, data/schema and integrations.
- Hotfix runbook requires incident linkage, independent review, protected deployment and back-promotion.
- Drift-monitoring procedure defines schedule, triage, ownership, reconciliation and escalation.
- Commands and screenshots contain no secrets or production session data.
- Runbooks are rehearsed in non-production and published in Confluence with owners and review dates.

SALDEV-1500-A10 — Define GTM test strategy and release evidence standard

Field	Value
Jira issue type	Story
Suggested owner	QA lead + Architect
Dependency	A06-A08
Indicative estimate	8 points
Delivery wave	Wave 2

User story

As QA and Architecture, we want risk-based test and evidence standards so that release decisions are consistent across code, configuration and integrations.

Acceptance criteria

- Test matrix defines minimum Apex/LWC/static/integration/regression/UAT coverage by change class.
- Permission/persona, CPQ/OmniStudio, integration and negative-path expectations are documented.
- Evidence schema from Section 9 is implemented in Jira templates and deployment reporting.
- Partial, blocked, skipped and not-applicable outcomes are distinguishable; only explicit passes satisfy a gate.
- Test-data ownership and masking/synthetic-data rules are approved.
- Monthly evidence audit procedure and sample size are defined.

SALDEV-1500-A11 — Pilot the operating model with one low-risk Salesforce change

Field	Value
Jira issue type	Story
Suggested owner	Delivery lead + Release manager
Dependency	A02-A10
Indicative estimate	8 points
Delivery wave	Wave 3

User story

As GTM Systems leadership, we want a controlled end-to-end pilot so that the process is proven before broader adoption.

Acceptance criteria

- Pilot ticket meets Definition of Ready and uses the approved repository, branch, PR and Jira workflow.
- Required CI checks and Salesforce validation pass; an intentional failure is demonstrated safely before correction.
- The same approved source lineage is promoted through INT, QA, UAT and the protected production path.
- Rollback is rehearsed in non-production and production smoke evidence is captured.
- Pilot meets every exit criterion in Section 16 and records time, failures, manual effort and user feedback.
- Critical gaps are resolved before scale-out; non-critical gaps have owners and dates.

SALDEV-1500-A12 — Train GTM roles and transition to business-as-usual governance

Field	Value
Jira issue type	Story
Suggested owner	Delivery lead + Platform & DevOps
Dependency	A11
Indicative estimate	8 points
Delivery wave	Wave 4

User story

As a GTM contributor, reviewer, tester or release manager, I want role-based training and support so that I can use the operating model correctly without relying on tribal knowledge.

Acceptance criteria

- Role-based guides and live sessions cover contributor/admin, reviewer, QA, business approver and release manager tasks.
- Each participant completes a practical exercise in a non-production repository/project.
- Support channel, office hours, escalation path and named tool owners are published.
- Legacy direct-deployment paths are disabled or documented as time-bound exceptions after pilot acceptance.
- Metrics dashboard and monthly control review begin with named owners.
- Operating model and Confluence pages receive final owner approval and next-review dates.

18. Acceptance-criteria traceability and ready-to-paste response

SALDEV-1500 acceptance item	Resolution	Evidence
What tool works best?	GitHub Enterprise Cloud as canonical repository; Gearset preferred greenfield Salesforce release layer; Copado approved when existing investment is decisive.	Sections 3-4; A01
What repository structure?	Domain-aligned repo portfolio plus standard Salesforce DX tree, ownership and content rules.	Sections 5-6; Appendix A; A03-A04
How will deployment/CI/CD work?	Ticket-to-production lifecycle, environment gates, validation evidence, approvals, rollback and drift controls.	Sections 7, 9-12; A05-A07, A09-A11
How will Jira tickets move?	Evidence-gated statuses with transition owners, required fields, rejection paths and automation boundaries.	Section 8; A08
What stories implement the plan?	Twelve Jira-ready stories covering decision, foundation, automation, pilot and adoption.	Sections 16-17
How is Confluence completed?	Controlled GTM Systems page tree with owners, reviews and linked runbooks.	Section 15; A09, A12

Ready-to-paste SALDEV-1500 comment

Jira comment: Completed the proposed GTM Systems DevOps and Jira operating model for SALDEV-1500. The recommendation is to use GitHub Enterprise Cloud as the canonical source repository, Jira as the authoritative work/approval record, and a Salesforce-aware release platform on top of Git. Gearset is the preferred greenfield option; Copado remains the approved alternative if Flywire's existing license, skills and operating investment make it lower risk. The document defines the repository portfolio and Salesforce DX folder structure, branch and pull-request controls, end-to-end deployment flow, Jira statuses and evidence gates, CI/CD quality checks, environment/production controls, rollback/hotfix/drift processes, RACI, Confluence page structure, rollout measures and twelve Jira-ready implementation stories. Before procurement, confirm that the transcript reference to "GitSense" means Gearset. No repositories, Jira workflows, Salesforce orgs or production pipelines were changed as part of this planning task.

Closure recommendation

SALDEV-1500 can be closed after: (1) this operating model is approved and published in the GTM Systems Confluence space; (2) the implementation epic and stories are created and linked; and (3) accountable owners accept A01 through A12. Tool purchase, configuration and pipeline execution remain implementation work and should not be represented as completed by this document alone.

Appendix A. Copy-ready repository tree

```

gtm-salesforce-core/
├── .github/
│   ├── CODEOWNERS
│   ├── pull_request_template.md
│   └── workflows/
│       ├── pr-validate.yml
│       ├── integration-promote.yml
│       └── release-evidence.yml
├── force-app/main/default/
│   ├── classes/
│   ├── flows/
│   ├── lwc/
│   ├── objects/
│   ├── permissionsets/
│   └── ...native Salesforce metadata folders
├── packages/<domain>/ # optional after dependency review
├── manifest/
│   ├── package.xml
│   ├── destructiveChangesPre.xml
│   └── destructiveChangesPost.xml
├── config/
│   └── project-scratch-def.json
├── scripts/
│   ├── ci/
│   └── release/
├── tests/
│   ├── integration/
│   └── ui/
├── data/seed/ # synthetic or masked definitions only
├── docs/
│   ├── adr/
│   ├── architecture/
│   ├── releases/
│   └── runbooks/
├── .forceignore
├── .gitignore
├── CONTRIBUTING.md
├── README.md
├── SECURITY.md
├── package.json
└── sfdx-project.json

```

Standard pull-request body

Field	Required content
Jira	SALDEV-####
Business outcome	What changes for which persona?
Scope	Components/files added, changed and deleted
Explicit exclusions	What is intentionally not included?
Risk/change class	Standard / Normal / High risk / Emergency
Validation	Job ID, target, component counts and test results
Manual steps	Before/after deployment steps and owners
Rollback	Revert/forward-fix/restoration path and decision point
Evidence	Screenshots, logs, QA/UAT links and known limitations

Copy/paste pull-request template

```
## Summary
Jira: SALDEV-####
Business outcome: <persona and measurable outcome>

## Scope and evidence
Added / changed / deleted: <components and files>
Explicit exclusions: <items intentionally omitted>
Validation: <target, job ID, component counts and result>
Tests: <level, run/pass/fail counts and omissions>
QA / UAT evidence: <links and approver>

## Risk and release
Change class: Standard | Normal | High risk | Emergency
Manual steps: <before/after steps and owner>
Rollback / forward fix: <action, decision point and owner>
Known limitations: <none or linked issue>
```

Appendix B. Sources and assumptions

Sources were accessed on 28 August 2026. Vendor features, editions, licensing and limits can change; procurement and security teams must confirm current contract terms before purchase. Vendor documentation supports capability statements, while the operating-model recommendation and trade-offs are an architecture judgment for the SALDEV-1500 context.

- [S1] [GitHub Docs — Deployments and environments](#)
- [S2] [GitHub Enterprise Cloud Docs — Managing protected branches](#)
- [S3] [GitHub Enterprise Cloud Docs — Establishing a governance framework](#)
- [S4] [Gearset Help Center — Continuous integration](#)
- [S5] [Gearset Help Center — Supported software integrations](#)
- [S6] [Gearset Help Center — Sample source-driven Salesforce workflow](#)
- [S7] [Copado Docs — Known limitations in Salesforce Source Format Pipelines](#)
- [S8] [Copado Docs — Structure of a user story](#)
- [S9] [Atlassian Support — What are Jira workflows?](#)
- [S10] [Atlassian Support — What are approvals?](#)
- [S11] [Salesforce Developers — Salesforce CLI command reference](#)
- [S12] [SALDEV-1500 Jira ticket](#)

Assumptions requiring confirmation

- “GitSense” means Gearset; if not, A01 must compare the intended product.
- GitHub Enterprise Cloud is acceptable under Flywire’s enterprise architecture and vendor-security policies.
- Jira edition and project type support the desired approval/automation behavior; if not, validators and explicit approval evidence will be used.
- Environment names shown are logical. The implementation stories must map them to Flywire’s actual Salesforce orgs without assuming aliases or access.
- CPQ/OmniStudio and other GTM configuration will be versioned only through supported, testable source representations; unsupported artifacts require documented manual steps or a specialized tool capability.
- This document does not approve a deployment, license purchase or external-system change.